

Business Intelligence & Sales Analytics Platform

End-to-end project report: Amazon web scraping → MySQL → Power BI. This production-style pipeline sources raw product data directly from Amazon (live market signals) and transforms it into a decision-ready executive dashboard. Tools used: Python (Scrapy, Selenium, Pandas), SQLAlchemy, MySQL, Power BI Desktop. The project covers three phases: Data Collection, Data Storage & SQL Analysis, and Power BI Reporting.

Project Overview — Purpose & Scope

Objective

Design and deliver a complete BI pipeline that ingests live Amazon product data and produces an interactive executive Power BI dashboard reflecting pricing, demand signals, and category performance.

Scope

Three phases: (1) Data engineering (scraping & storage), (2) Data transformation (SQL analysis), (3) Business reporting (Power BI dashboards).

Why live scraping?

Using real Amazon data ensures analyses reflect actual market pricing, customer demand signals, category mix and dynamic discounts—not a static, pre-packaged dataset.

Phase 1 — Data Collection Architecture (Scrapy + Selenium)



Scrapy Spider (amazon_products)

Structured Scrapy project handles bulk scraping across 20 product categories (Technology, Furniture, Home Appliances, Apparel). Runs at low concurrency with randomized 3–9s delays to mimic human browsing and reduce blocking risk.



Selenium Fallback

Standalone Selenium script drives a real Chrome browser via webdriver-manager for JavaScript-rendered pages that require real browser rendering to capture content.



Anti-Block Middleware

Custom downloader middleware rotates realistic User-Agent profiles, detects CAPTCHA/blocked pages, saves debug HTML for selector troubleshooting, and isolates blocked responses for analysis.

selenium_scraper.py > ...

```

1  import os
2  import re
3  import csv
4  import time
5  import random
6  import datetime
7  import logging
8  (module) selenium
9  from selenium import webdriver
10 from selenium.webdriver.chrome.service import Service
11 from selenium.webdriver.chrome.options import Options
12 from selenium.webdriver.common.by import By
13 from selenium.webdriver.support.ui import WebDriverWait
14 from selenium.webdriver.support import expected_conditions as EC
15 from selenium.common.exceptions import TimeoutException, NoSuchElementException
16 from webdriver_manager.chrome import ChromeDriverManager
17
18 logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
19 logger = logging.getLogger(__name__)
20
21
22
23 KEYWORDS = ["laptop", "wireless earbuds", "office chair", "smart watch", "bluetooth speaker"]
24 PAGES_PER_KEYWORD = 2
25 HEADLESS = True
26 MIN_DELAY = 4
27 MAX_DELAY = 9
28 # -----
29
30
31 USER_AGENTS = [
32     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
33     "(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36",
34     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
35     "(KHTML, like Gecko) Chrome/123.0.0.0 Safari/537.36",
36     "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:125.0) Gecko/20100101 Firefox/125.0",
37 ]
38
39

```

New_Query_1781425329601.sql

generate_sales_data.py

selenium_scraper.py

selenium_scraper.py > ...

```
38
39
40 > def build_driver():...
63
64
65 def to_float(text):
66     if not text:
67         return None
68     match = re.search(r"[\d,]+\.\d*", text.replace(",", ""))
69     return float(match.group()) if match else None
70
71
72 def to_int(text):
73     if not text:
74         return None
75     text = text.upper().replace(",", "")
76     multiplier = 1
77     if "K" in text:
78         multiplier = 1000
79     elif "M" in text:
80         multiplier = 1_000_000
81     text = text.replace("M", "")
82     match = re.search(r"[\d.]+", text)
83     return int(float(match.group()) * multiplier) if match else None
84
85
86
87 > def parse_results_page(driver, keyword):...
189
190
191 > def main():...
252
253
254 if __name__ == "__main__":
255     main()
256
```

Phase 1 — Data Model & Export

Each product record captures ASIN, title, brand, category, current price, original (MRP) price, discount %, average star rating, review count, 'bought last month' demand signal, Prime eligibility, and UTC timestamp. A custom Scrappy pipeline performs inline type conversion (prices → float, review counts parse K/M suffixes) and exports timestamped UTF-8 CSV files.

OPEN EDITORS

AMAZON_SALES_ANALYTICS

.vscode

launch.json

settings.json

mnt

output

venv

init.py

amazon_products.py

analyze_data.py

generate_sales_data.py

items.py

middlewares.py

mysqllocal.session.sql

pipelines.py

scrapy.cfg

selenium_scraper.py

settings.py

SQL_PUSH.py

generate_sales_data.py > ...

```

1 import sys
2 import glob
3 import random
4 import datetime
5 import pandas as pd
6 import numpy as np
7
8 # ---- CONFIG ----
9 NUM_ORDERS = 5000 # number of order line-items to generate
10 START_DATE = datetime.date(2023, 1, 1)
11 END_DATE = datetime.date(2025, 12, 31)
12
13 REGIONS = {
14     "East": ["New York", "Boston", "Philadelphia"],
15     "West": ["Los Angeles", "San Francisco", "Seattle"],
16     "Central": ["Chicago", "Dallas", "Austin"],
17     "South": ["Miami", "Atlanta", "Houston"],
18 }
19
20 SEGMENTS = ["Consumer", "Corporate", "Home Office"]
21
22 FIRST_NAMES = ["James", "Mary", "Robert", "Patricia", "John", "Jennifer",
23               "Michael", "Linda", "David", "Elizabeth", "Sara", "Aman",
24               "Priya", "Rahul", "Neha", "Karan", "Riya", "Vikram"]
25 LAST_NAMES = ["Smith", "Johnson", "Williams", "Brown", "Jones", "Garcia",
26              "Miller", "Davis", "Sharma", "Patel", "Khan", "Gupta",
27              "Singh", "Verma"]
28
29 # Map scraped "category" keywords -> business Category / Sub-Category
30 CATEGORY_MAP = {
31     "laptop": ("Technology", "Computers"),
32     "smartphone": ("Technology", "Phones"),
33     "headphones": ("Technology", "Audio"),
34     "wireless earbuds": ("Technology", "Audio"),
35     "smart watch": ("Technology", "Wearables"),
36     "tablet": ("Technology", "Computers"),
37     "office chair": ("Furniture", "Chairs"),
38     "desk": ("Furniture", "Tables"),
39     "monitor": ("Technology", "Computer Accessories"),
40     "keyboard": ("Technology", "Computer Accessories")

```


EXPLORER

> OPEN EDITORS

AMAZ...
> .vscode
 launch.json
 settings.json
> mnt
> output
> venv
 __init__.py
 amazon_products.py
 analyze_data.py
 generate_sales_data.py
 items.py
 middlewares.py
 mysqllocal.session.sql
 pipelines.py
 scrapy.cfg
 selenium_scraper.py
 settings.py
 SQL_PUSH.py

sales
analyze_data.py
generate_sales_data.py X

generate_sales_data.py > ...
37 "office chair": ("Furniture", "Chairs"),
38 "desk": ("Furniture", "Tables"),
39 "monitor": ("Technology", "Computer Accessories"),
40 "keyboard": ("Technology", "Computer Accessories"),
41 "printer": ("Technology", "Office Machines"),
42 "kitchen appliance": ("Home & Kitchen", "Appliances"),
43 "vacuum cleaner": ("Home & Kitchen", "Appliances"),
44 "air fryer": ("Home & Kitchen", "Appliances"),
45 "coffee maker": ("Home & Kitchen", "Appliances"),
46 "running shoes": ("Apparel", "Footwear"),
47 "backpack": ("Apparel", "Bags"),
48 "fitness tracker": ("Technology", "Wearables"),
49 "gaming console": ("Technology", "Gaming"),
50 "camera": ("Technology", "Cameras"),
51 "bluetooth speaker": ("Technology", "Audio"),
52 }
53
54
55 def load_products(path=None):
56 if path is None:
57 files = sorted(glob.glob("output/amazon_selenium*.csv")) + \
58 sorted(glob.glob("output/amazon_products*.csv"))
59 if not files:
60 raise FileNotFoundError("No scraped product CSV found in output/.")
61 path = files[-1]
62 print(f"Using product catalog: {path}")
63
64 df = pd.read_csv(path)
65 df = df.dropna(subset=["title", "price"])
66 df["price"] = pd.to_numeric(df["price"], errors="coerce")
67 df = df.dropna(subset=["price"])
68 df = df[df["price"] > 0]
69 return df.reset_index(drop=True)
70
71
72 def random_date(start, end):
73 delta = (end - start).days
74 return start + datetime.timedelta(days=random.randint(0, delta))
75
76

EXPLORER

salesanalyze_data.pygenerate_sales_data.py X

OPEN EDITORS

AMAZON_SALES_ANALYT...

.vscode

{ launch.json

{ settings.json

mnt

output

venv

init.py

amazon_products.py

analyze_data.py

generate_sales_data.py

items.py

middlewares.py

mysqllocal.session.sql

pipelines.py

scrapy.cfg

selenium_scraper.py

settings.py

SQL_PUSH.py

OUTLINE

TIMELINE

generate_sales_data.py > ...

71

72 def random_date(start, end):

73 delta = (end - start).days

74 return start + datetime.timedelta(days=random.randint(0, delta))

75

76

77 def main():

78 path = sys.argv[1] if len(sys.argv) > 1 else None

79 products = load_products(path)

80

81 # Build a product catalog with business category mapping

82 catalog = []

83 for idx, row in products.iterrows():

84 kw = str(row["category"]).strip().lower()

85 cat, subcat = CATEGORY_MAP.get(kw, ("General", "Misc"))

86 catalog.append({

87 "Product ID": f"PROD-{idx+1:04d}",

88 "Product Name": row["title"][:80],

89 "Category": cat,

90 "Sub Category": subcat,

91 "Unit Price": round(float(row["price"]), 2),

92 })

93 catalog_df = pd.DataFrame(catalog)

94 print(f"Catalog built with {len(catalog_df)} products across "

95 f"{catalog_df['Category'].nunique()} categories and "

96 f"{catalog_df['Sub Category'].nunique()} sub-categories.")

97

98 # Generate customers

99 num_customers = 400

100 customers = []

101 for i in range(num_customers):

102 name = f"{random.choice(FIRST_NAMES)} {random.choice(LAST_NAMES)}"

103 region = random.choice(list(REGIONS.keys()))

104 state_city = random.choice(REGIONS[region])

105 customers.append({

106 "Customer ID": f"CUST-{i+1:04d}",

107 "Customer Name": name,

108 "Segment": random.choice(SEGMENTS),

109 "Region": region,

110 "City": state_city,

EXPLORER

> OPEN EDITORS

AMAZON_SALES_ANALYTICS

.vscode

launch.json

settings.json

mnt

output

venv

__init__.py

amazon_products.py

analyze_data.py

generate_sales_data.py

items.py

middlewares.py

mysqllocal.session.sql

pipelines.py

scrapy.cfg

selenium_scraper.py

settings.py

SQL_PUSH.py

OUTLINE

TIMELINE

sales

analyze_data.py

generate_sales_data.py

generate_sales_data.py > ...

77 def main():

108 "Segment": random.choice(SEGMENTS),

109 "Region": region,

110 "City": state_city,

111 "State": region, # simplified; could map to real states

112 "Country": "United States",

113

114 })

115

116 customers_df = pd.DataFrame(customers)

117

118 # Generate orders

119 orders = []

120 for i in range(NUM_ORDERS):

121 product = catalog_df.sample(1).iloc[0]

122 customer = customers_df.sample(1).iloc[0]

123

124 order_date = random_date(START_DATE, END_DATE)

125 ship_date = order_date + datetime.timedelta(days=random.randint(1, 7))

126

127 quantity = random.randint(1, 6)

128 # Discounts: simulate occasional promotions

129 discount = random.choice([0, 0, 0, 0.05, 0.1, 0.15, 0.2, 0.3])

130

131 unit_price = product["Unit Price"]

132 gross_sales = unit_price * quantity

133 sales = round(gross_sales * (1 - discount), 2)

134

135 # Profit margin varies by category (simulate realistic margins)

136 margin_lookup = {

137 "Technology": 0.18,

138 "Furniture": 0.12,

139 "Home & Kitchen": 0.22,

140 "Apparel": 0.30,

141 "General": 0.15,

142 }

143 base_margin = margin_lookup.get(product["Category"], 0.15)

144

145 margin = base_margin + np.random.normal(0, 0.08)

146 profit = round(sales * margin, 2)

147

148 orders.append({

EXPLORER

> OPEN EDITORS

AMAZ...

.vscode

launch.json

settings.json

mnt

output

venv

init.py

amazon_products.py

analyze_data.py

generate_sales_data.py

items.py

middlewares.py

mysqllocal.session.sql

pipelines.py

scrapy.cfg

selenium_scraper.py

settings.py

SQL_PUSH.py

> OUTLINE

> TIMELINE

sales

analyze_data.py

generate_sales_data.py

generate_sales_data.py > ...

77 def main():

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

orders.append({

"Order ID": f"ORD-{i+1:06d}",

"Order Date": order_date.isoformat(),

"Ship Date": ship_date.isoformat(),

"Customer ID": customer["Customer ID"],

"Customer Name": customer["Customer Name"],

"Segment": customer["Segment"],

"Product ID": product["Product ID"],

"Product Name": product["Product Name"],

"Category": product["Category"],

"Sub Category": product["Sub Category"],

"Sales": sales,

"Quantity": quantity,

"Discount": discount,

"Profit": profit,

"Country": customer["Country"],

"State": customer["State"],

"City": customer["City"],

"Region": customer["Region"],

})

orders_df = pd.DataFrame(orders)

out_path = "output/sales_transactions.csv"

orders_df.to_csv(out_path, index=False, encoding="utf-8-sig")

print(f"Saved {len(orders_df)} order records to {out_path}")

catalog_df.to_csv("output/dim_products.csv", index=False, encoding="utf-8-sig")

customers_df.to_csv("output/dim_customers.csv", index=False, encoding="utf-8-sig")

print("Also saved output/dim_products.csv and output/dim_customers.csv")

if __name__ == "__main__":

main()

1

3

Git Graph

127.0.0.1

amazon_analytics

Ln 1, Col 1

Spaces: 4

UTF-8

LF

Made with GAMMA

Phase 2 — Storage & Synthetic Sales Generation (MySQL + SQLAlchemy)

Cleaned product CSVs were loaded into MySQL (amazon_analytics) using pandas + SQLAlchemy to avoid encoding issues. A supplementary Python script (generate_sales_data.py) used the scraped product catalog to generate a realistic 5,000-row transactional sales dataset with Order Date, Customer ID, Segment, Region, Sales, Discount, and Profit—creating a proper analytics-ready fact table for SQL and Power BI.

Phase 2 — SQL Techniques & Key Queries

SQL techniques used to extract business insights:



Window Functions

RANK() OVER (PARTITION BY category) for top-5 revenue products; SUM() OVER for running totals; LAG() for month-over-month growth.



CTEs & Multi-step Logic

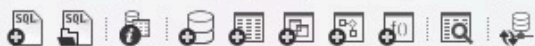
CTEs for customer segmentation cohorts, ABC inventory classification (A = top 80% revenue), and multi-stage analyses.



Profitability Analysis

Discount-band grouping quantified correlation between deep discounts (>20%) and negative profit margins; results fed into dashboard alerts.

Example: Month-over-month revenue growth using LAG() (query applied to the generated sales fact table to compute pct_growth by month).



Navigator

SCHEMAS

Filter objects

- amazon_analytics
 - Tables
 - dim_customers
 - dim_products
 - products
 - sales
 - Columns
 - Order ID
 - Order Date
 - Ship Date
 - Customer ID
 - Customer Name
 - Segment

Administration Schemas

Information

No object selected

dim_customers

SQL File 10*

SQL File 11* x



Don't Limit

```
1  -- Discount impact analysis - does higher discount actually hurt profit?
2  •  SELECT
3      CASE
4          WHEN Discount = 0 THEN 'No Discount'
5          WHEN Discount <= 0.1 THEN 'Low (≤10%)'
6          WHEN Discount <= 0.2 THEN 'Medium (≤20%)'
7          ELSE 'High (>20%)'
8      END AS discount_band,
9      COUNT(*) AS num_orders,
10     AVG(Profit) AS avg_profit,
11     AVG(Sales) AS avg_sales,
12     ROUND(AVG(Profit)/AVG(Sales)*100,2) AS avg_margin_pct
13 FROM sales
14 GROUP BY discount_band
15 ORDER BY avg_margin_pct DESC;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

	discount_band	num_orders	avg_profit	avg_sales	avg_margin_pct
▶	Low (≤10%)	1273	14850.796535742345	89056.18641005507	16.68
	High (>20%)	635	11273.983511811026	67972.29333858269	16.59
	No Discount	1814	16204.022337375944	98499.13377067221	16.45
	Medium (≤20%)	1278	13248.618528951512	81757.6169170578	16.2

Result 2 x

Read Only

SQL Query 2

The screenshot shows a SQL IDE interface with a menu bar (File, Edit, View, Query, Database, Server, Tools, Scripting, Help) and a toolbar. The left sidebar contains a 'Navigator' pane with a 'SCHEMAS' tree showing the 'amazon_analytics' database and its tables: 'dim_customers', 'dim_products', 'products', and 'sales'. The 'Columns' section for 'sales' lists 'Order ID', 'Order Date', 'Ship Date', 'Customer ID', 'Customer Name', and 'Segment'. Below the navigator are 'Administration' and 'Schemas' tabs, and an 'Information' pane stating 'No object selected'.

The main editor displays a SQL query in a file named 'SQL File 10*':

```
1  -- Loss-making orders investigation (filtering + business insight)
2  • SELECT `Order ID`, `Product Name`, Category, Sales, Discount, Profit,
3         ROUND(Profit / Sales * 100, 2) AS margin_pct
4  FROM sales
5  WHERE Profit < 0
6  ORDER BY Profit ASC
7  LIMIT 20;
```

Below the query editor is a 'Result Grid' showing 20 rows of data. The columns are Order ID, Product Name, Category, Sales, Discount, Profit, and margin_pct. The data is sorted by Profit in ascending order.

Order ID	Product Name	Category	Sales	Discount	Profit	margin_pct
ORD-004781	Garmin epix Pro (Gen 2) Sapphire Edition, 51mm...	Technology	263519.45	0	-25336.09	-9.61
ORD-001678	Acer Nitro V Gaming Laptop Intel Core i5-1342...	Technology	245314.05	0.2	-14666.27	-5.98
ORD-000844	Amazfit Active Max Smart Watch 1.5" AMOLED ...	Technology	92850.89	0.05	-13601.97	-14.65
ORD-000850	Samsung Galaxy Watch 8 (2025) 44mm Bluetoo...	Technology	156384.77	0.15	-12795.59	-8.18
ORD-004262	GEEKOM GeekBook X16 Pro 2.8 lbs Laptop, 16" ...	Technology	361957.74	0.3	-11939.26	-3.3
ORD-004303	Samsung Galaxy Buds 4 Pro (2026) AI True Wir...	Technology	109932.72	0.2	-11140.3	-10.13
ORD-000563	HON Convergence Ergonomic Office Chair with ...	Furniture	101765.4	0	-11137.62	-10.94
ORD-000507	HON Ignition 2.0 Mid-Back Ergonomic Office Ch...	Furniture	190889.05	0.15	-10680	-5.59
ORD-003808	GEEKOM GeekBook X16 Pro 2.8 lbs Laptop, 16" ...	Technology	517082.48	0.2	-8411.12	-1.63
ORD-002280	Amazon Basics Ergonomic Executive Office Desk...	Furniture	112400.18	0.15	-8390.84	-7.47
ORD-001106	ASUS ROG Strix G16 (2025) Gaming Laptop, 16"	Technology	609934.29	0.05	-7046.71	-1.16

Below the result grid is an 'Output' pane showing the execution log. It contains two entries, both successful, showing the query execution time and the number of rows returned.

#	Time	Action	Message	Duration / Fetch
20	13:57:29	SELECT 'Order ID', 'Product Name', Category, Sales, Discount, Profit, ROUND(...	20 row(s) returned	0.015 sec / 0.000 s
21	13:57:31	SELECT 'Order ID', 'Product Name', Category, Sales, Discount, Profit, ROUND(...	20 row(s) returned	0.031 sec / 0.000 s

SQL Query 3

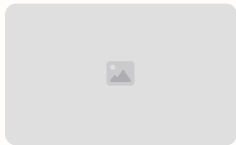
The screenshot shows a SQL IDE interface. On the left, a tree view displays the database schema for 'amazon_analytics', including tables 'dim_customers', 'dim_products', 'products', and 'sales', along with their columns. The main editor displays a SQL query to find the top 5 products per category by revenue. The query uses a CTE 'product_sales' to calculate total sales per category and product, then ranks them. The results are shown in a table at the bottom.

```
1  -- Top 5 products per category by revenue (PARTITION BY + RANK)
2  WITH product_sales AS (
3      SELECT Category, `Product Name`, SUM(Sales) AS total_sales
4      FROM sales
5      GROUP BY Category, `Product Name`
6  )
7  SELECT *
8  FROM (
9      SELECT *,
10         RANK() OVER (PARTITION BY Category ORDER BY total_sales DESC) AS rnk
11      FROM product_sales
12  ) ranked
13  WHERE rnk <= 5;
```

	Category	Product Name	total_sales	rnk
▶	Furniture	HON Ignition 2.0 Mid-Back Ergonomic Office Ch...	12841218.560000002	1
	Furniture	HON Convergence Ergonomic Office Chair with ...	7997064.349999999	2
	Furniture	Amazon Basics Ergonomic Executive Office Desk...	6044264.29	3
	Furniture	Serta Bryce Executive Office Chair - Ergonomic ...	5928007.079999997	4
	Furniture	Serta Garret Executive Office Chair, Ergonomic ...	5688964.020000005	5
	Technology	GEEKOM GeekBook X16 Pro 2.8 lbs Laptop, 16" ...	69153318.27999994	1
	Technology	Acer Nitro V 16S AI Gaming Laptop AMD Ryze...	38030043.650000006	2
	Technology	ASUS ROG Strix G16 (2025) Gaming Laptop, 16"...	36262198.919999994	3

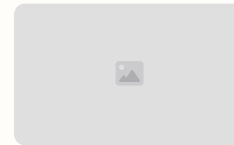
Phase 3 — Power BI Report & Modelling

The cleaned and transformed dataset was loaded into Power BI Desktop via Text/CSV. A DateTable was created in DAX and marked as a Date Table. Relationships were defined between sales fact and dimension tables (products, customers) to implement a star schema. The final report includes two pages: Executive Summary and Profitability Deep-Dive.



Executive Summary (Page)

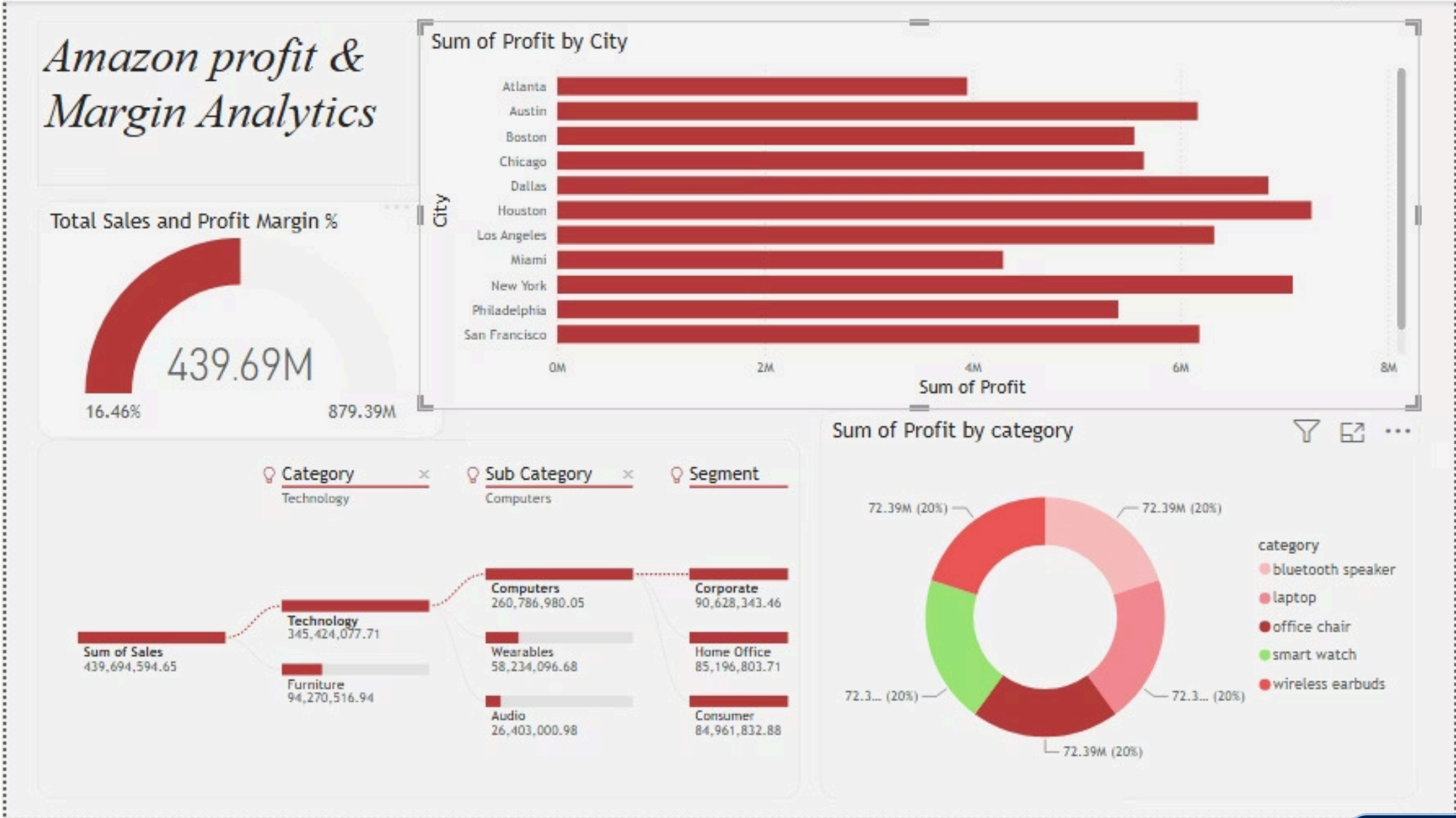
KPI cards: Total Sales 439.69M, Profit 72.39M, Margin 16.46%, Orders 5K; area chart Sales vs Profit; USA map by city; segment donut chart. Key headline: Sales YoY growth 49.64%.



Profitability Deep-Dive (Page)

Profit by city bar chart, profit by category donut, Decomposition Tree: Total Sales → Category → Sub-Category. Findings show Technology dominance and city-level concentration risks.

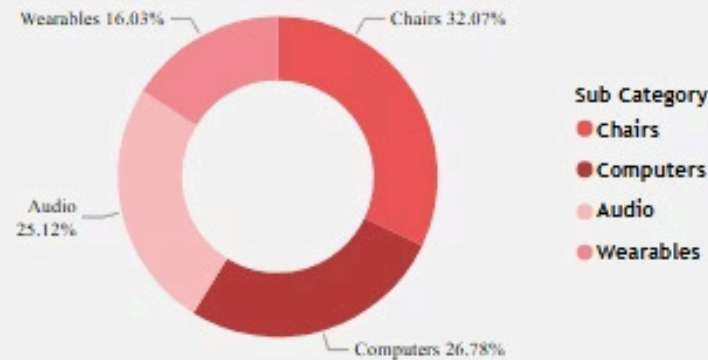
Power BI Dashboard 3: Profit & Margin Analytics



Power BI Dashboard 2: Customer & Bottleneck Analytics

Amazon Customer & Bottleneck Analytics

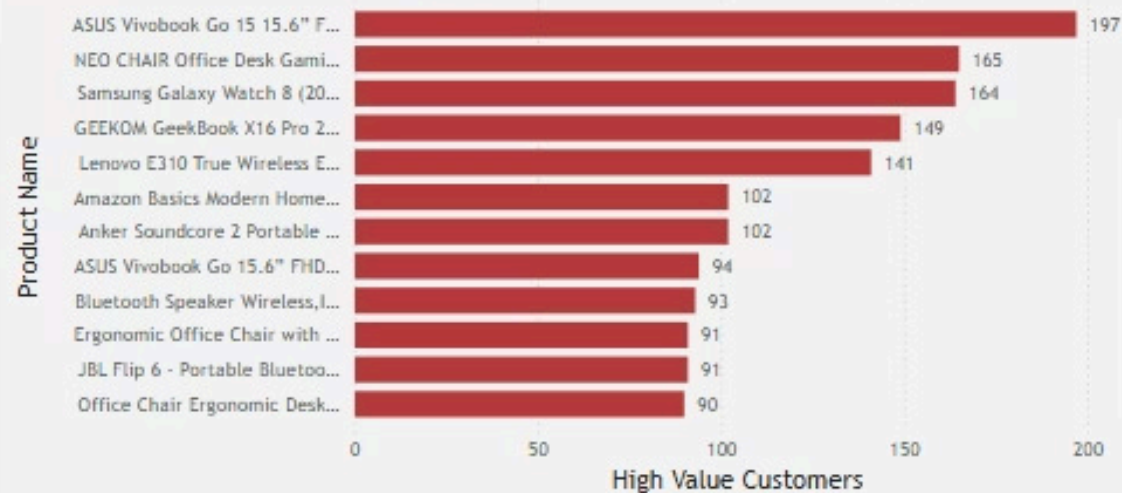
Repeat Customers by Sub Category



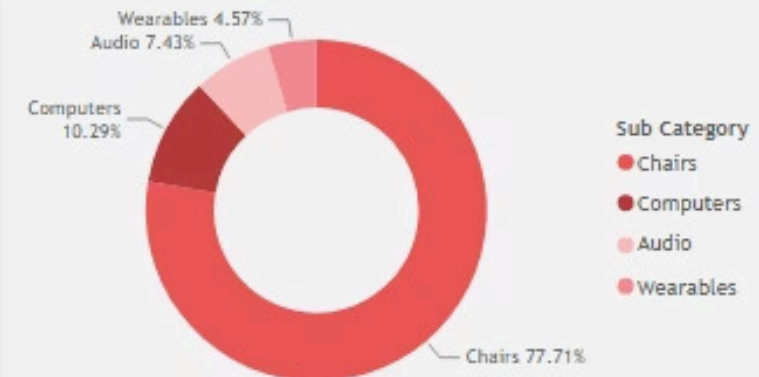
Loss Making Orders by City



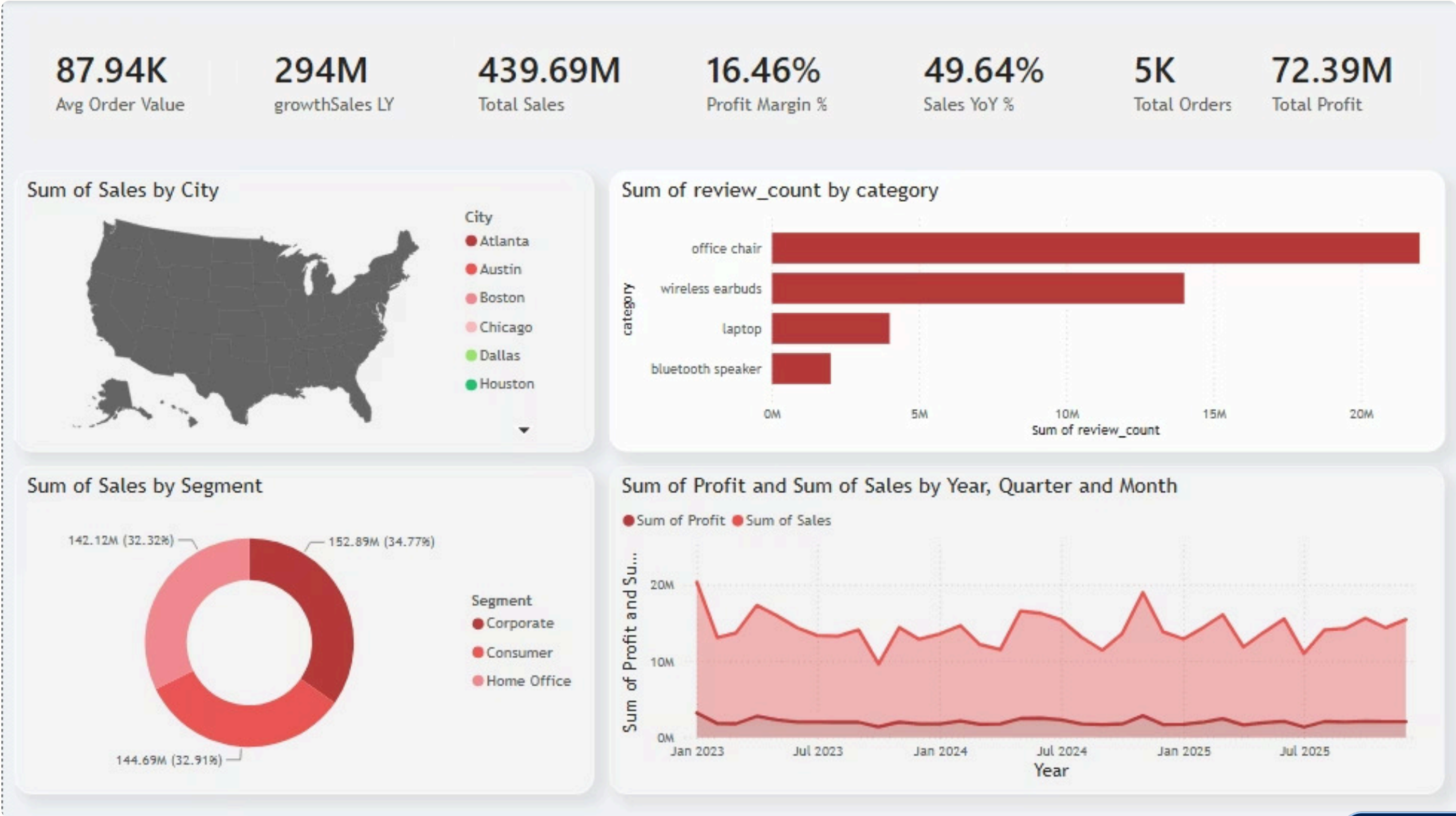
High Value Customers *** Product Name



Loss Making Orders by Sub Category



Power BI Dashboard 1: Executive Summary



DAX Measures & Analytical Logic

Core DAX measures developed to power dynamic KPIs and slicers:

- Profit Margin %: `DIVIDE([Total Profit], [Total Sales])` — surfaces margin compression even when raw profit looks healthy.
- Sales YoY %: `DIVIDE([Total Sales] - CALCULATE([Total Sales], SAMEPERIODLASTYEAR(DateTable[Date])), [Sales LY])` — requires proper DateTable.
- Customer Tier: `SWITCH(TRUE(), ...)` — classifies customers into High/Medium/Low based on spend relative to average for segmentation analysis.
- Loss Making Orders: `CALCULATE(COUNTROWS(sales), sales[Profit] < 0)` — quantifies discount-driven margin erosion immediately.

Key Business Findings

Revenue Concentration

Technology dominates revenue (345M, ~79% of total). Within Technology, Computers account for 260M (major share), creating category concentration risk.

Profit vs. Discount

Discounts >20% consistently correlate with negative profit margins across categories — confirmed by both SQL and DAX analyses.

Geographic Risk

Seattle, Austin, and New York are top three profit-generating cities; regional disruption could disproportionately impact performance.

Segment Dynamics

Consumer (34.77%), Home Office (32.91%), Corporate (32.32%) are near-equal splits by revenue, but Corporate orders have higher AOV — a lever for growth prioritization.

Demand Signals

'Bought last month' counts for high-rated (4.5+) products in Wearables and Audio categories far exceed Furniture — indicating stronger organic demand momentum in tech accessories vs big-ticket items.

Skills Demonstrated & Final Thoughts

1 Web Scraping

Scrapy project with custom middlewares, Selenium fallback, anti-bot mitigations, incremental UTF-8 CSV export pipeline.

2 Python / Pandas

Data cleaning, type normalization, EDA, SQLAlchemy ETL, synthetic sales generation to create realistic business distributions for analytics.

3 SQL

CTEs, window functions (RANK, LAG, running totals), cohort and segmentation logic, ABC analysis, discount-impact queries used to surface actionable insights.

4 Power BI

Star-schema modelling, DateTable, DAX measures (time-intelligence, SWITCH segmentation, DIVIDE), KPI cards, Decomposition Tree, geographic mapping for executive reporting.

5 Business Impact

Identified discount-margin correlation, geographic concentration risk, segment AOV differences, and category-level demand signals — enabling prioritized strategic actions.

This end-to-end project demonstrates an operational BI pipeline from raw data sourcing to decision-ready visual reporting. Next steps: refine anti-blocking strategies for long-term data collection, implement alerting for discount-driven losses, and prioritize growth initiatives targeting Corporate accounts and high-margin product lines.